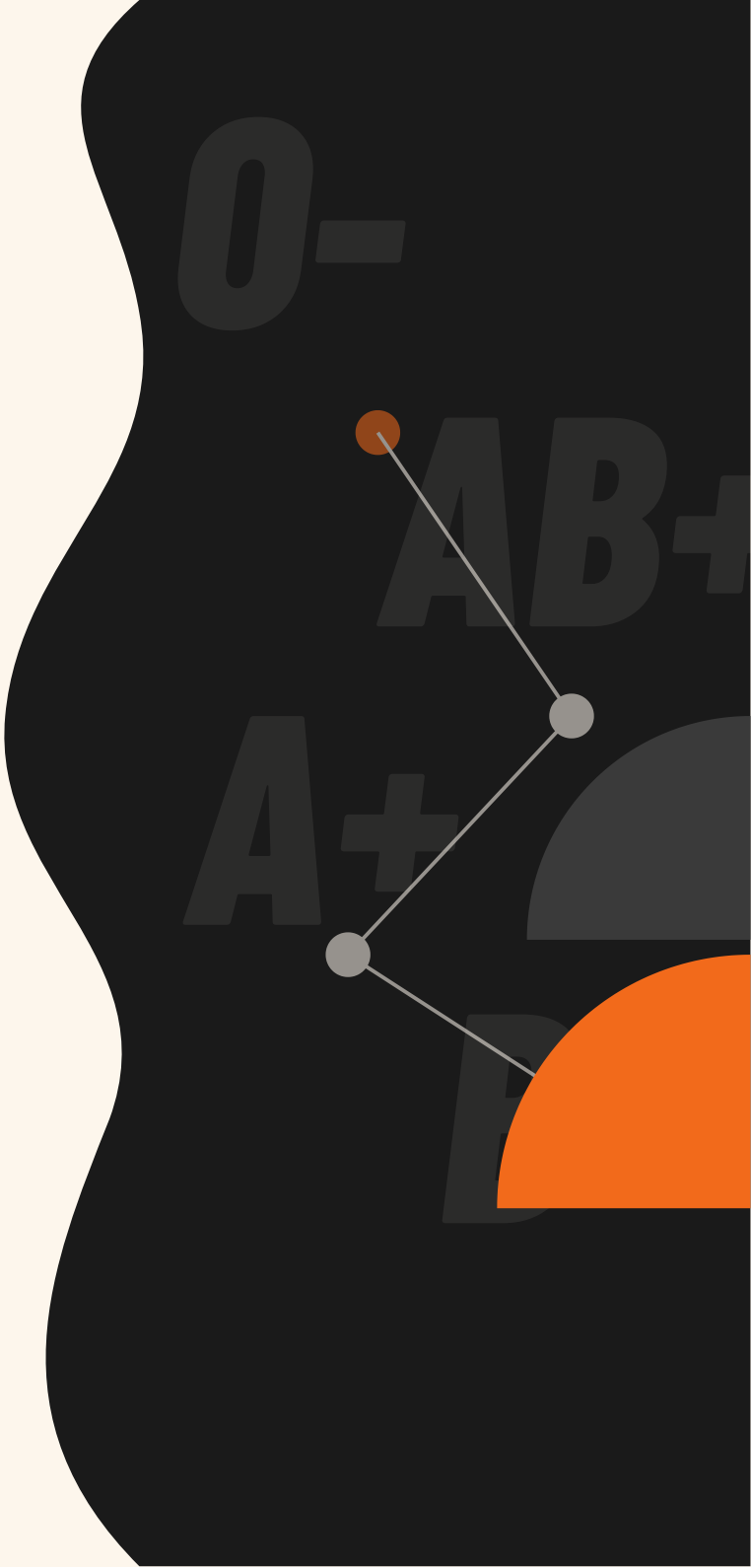
CESAR
school

Rota Vital

Projeto Integrador · Unidade 2

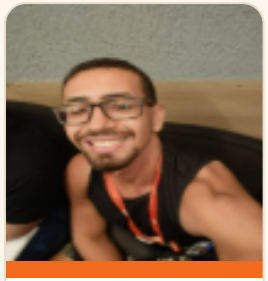
Algoritmos que dão inteligência à gestão e distribuição de hemocomponentes: compatibilidade ABO/Rh, FEFO, índice por hash e roteirização — implementados em C/C++ e traduzidos, comentados, para Java.



Equipe Rota Vital · Programação Orientada a Objetos e Estruturas de Dados
Recife, novembro de 2026 · dados 100% sintéticos

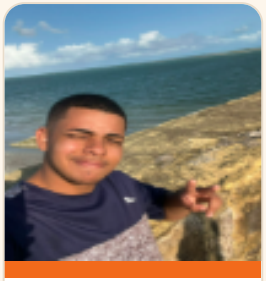
01 QUEM FEZ

Equipe



Ewerton Guilherme da Silva

Product Owner



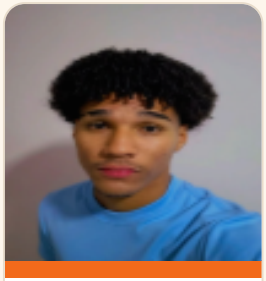
Pablo Arthur Eustáquio de Lima

Backend Developer



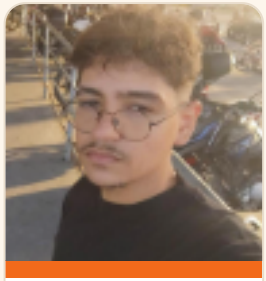
João Ricardo Alves de Brito

Backend Developer



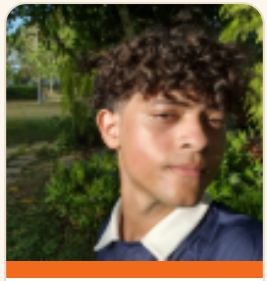
Lucas Aprigio dos Santos

Backend Developer



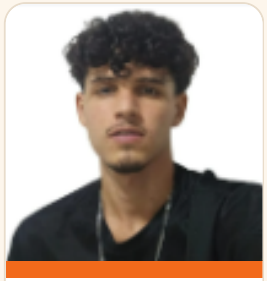
Saulo Eduardo Almeida dos Santos

Backend Developer



Thiago Cardozo da Conceição

Frontend / UI Designer



Eloi de Lima Sousa

Frontend / UI Designer

Sumário

02 O problema e a visão do sistema

03 Método da tradução comentada C/C++ → Java

04 Compatibilidade ABO/Rh — matriz

05 Priorização FEFO — lista com inserção ordenada

06 Índice de estoque — hash com encadeamento

07 Índice por id — hash com sondagem linear

08 Fila de pedidos — FIFO encadeada

09 Alocação integrada — o coração do sistema

10 Grafo — matriz de adjacência

11 Roteirização — vizinho mais próximo

12 Ordenação — os seis métodos do semestre

13 Busca — sequencial, binária iterativa e recursiva

14 Quadro-resumo de complexidade

15 Dicionário geral de tradução C/C++ → Java

16 Testes e evidências de execução

17 Atendimento aos critérios e à regra geral

18 Como executar · limites e cuidados

02 FICHA DO PROJETO

O problema e a visão do sistema

A rede de sangue precisa garantir **o componente certo** (compatível e dentro da validade), **no lugar certo**, **no tempo certo** e **na temperatura certa**. Falhas geram desabastecimento, descarte por vencimento e risco ao paciente. O fluxo inspirado na Hemorrede/SUS é: coleta e doação → hemocentro de processamento → estoque → hospitais.

Do pedido à entrega: o fluxo que implementamos

1 · Fila FIFO
Requisições hospitalares entram na ordem de chegada.

→

2 · Matriz ABO/Rh
8 consultas O(1) dizem quais tipos doadores servem.

→

3 · Hash
(componente, tipo) leva direto à lista daquela combinação.

→

4 · FEFO
Descarta vencidas e retira a compatível que vence primeiro (tudo ou nada).

→

5 · Grafo
Matriz de adjacência com o tempo de cada estrada.

→

6 · Vizinho + próximo
Rota de cada pedido, janela de tempo e cadeia fria.

🩸 Componente certo

Matriz de compatibilidade (hemácias/plaquetas e plasma/crioprecipitado com regra invertida) + descarte automático de vencidas antes de qualquer alocação.

⏱ Tempo certo

FEFO faz a bolsa mais próxima do vencimento sair primeiro; a rota verifica a janela de cada hospital e emite alerta quando ela é violada.

❄ Temperatura certa

Limite simulado de 120 min de caixa térmica; cada parada é marcada como "cadeia fria OK" ou "RISCO".

Mesmos algoritmos do sistema HemoTrack. O HemoTrack (Spring Boot) usa, no pacote `br.com.hemotrack.algoritmo`, exatamente estes quatro algoritmos: a mesma matriz 8×8, a mesma chave de hash (`componente × 8 + tipo`, `% 11`), a mesma regra de alocação (compatível que vence primeiro, tudo ou nada) e a mesma rota por vizinho mais próximo. Este documento é a versão C/C++ e a tradução comentada deles.

Onde cada algoritmo da disciplina aparece

Assunto (aula)	Estrutura no Rota Vital	Para que serve no problema
Recursão (02)	Busca binária recursiva, Quick e Merge Sort	Relatório "o que vence nos próximos 7 dias" e ordenação de relatórios.
Listas, pilhas e filas (03)	Lista FEFO encadeada · fila FIFO de pedidos	Estoque sempre ordenado por validade · atendimento justo por ordem de chegada.
Ponteiros (03.1)	`NoBolsa*`, `EntradaEstoque*`, parâmetros de saída	Encadeamento manual em C++; vira referência em Java.

Assunto (aula)	Estrutura no Rota Vital	Para que serve no problema
Árvores / grafos (04–05)	Grafo de 6 vértices (matriz de adjacência) + vizinho mais próximo com pilha	Rota do hemocentro até o hospital de cada pedido.
Busca e hash (06)	Hash com encadeamento + hash com sondagem linear · busca sequencial e binária	Achar o estoque de um tipo e uma bolsa por id em O(1) médio.
Ordenação (07–08)	Bubble, Selection, Insertion, Shell, Quick, Merge	Relatórios FEFO; comparação de custo real (contador de comparações).

Método da tradução comentada

A disciplina avalia o **raciocínio**, não só o programa rodando. Por isso escrevemos primeiro a versão C/C++ (structs, ponteiros, `new/delete`, funções que recebem a estrutura por ponteiro) e só depois traduzimos para Java orientado a objetos — mantendo **o mesmo algoritmo, os mesmos nomes de variáveis e a mesma saída**. A tradução é verificável: os dois programas imprimem exatamente o mesmo texto.

① Bloco de cabeçalho

Todo arquivo Java abre com um bloco **"C++ → Java"** que mostra a declaração original em C++ e explica o que mudou na versão orientada a objetos.

② Comentário na linha

Toda linha cuja forma mudou leva `// C++: ...` com a linha original, por exemplo `atual = atual.prox; // C++: atual = atual->prox;`

③ Mesma lógica, mesmos nomes

`fefoInserir` → `ListaFefo.inserir`, `hashBuscar` → `HashEstoque.buscar`. Os laços, condições e o desempate são os mesmos.

As quatro regras de tradução que seguimos

1. A estrutura com suas funções vira uma classe

Em C++, `struct HashEstoque` e as funções `hashBuscar(HashEstoque* h, ...)` ficam separadas. Em Java, os dados e as operações ficam juntos na classe; o primeiro parâmetro `h` some porque passa a ser o `this` implícito.

2. Ponteiro vira referência; `delete` some

`NoBolsa* prox` vira `No prox`, `->` vira `.` e `NULL` vira `null`. Não existe `delete`: quando nenhum objeto aponta mais para um nó, o coletor de lixo o libera. Por isso `estoqueLiberar()` não tem equivalente em Java.

3. Parâmetro de saída vira retorno

`bool dequeue(FilaPedidos* f, Pedido* saida)` escreve no ponteiro. Java não tem ponteiro de saída, então `dequeue()` devolve o `Pedido` ou `null`. A cabeça de lista retornada por `fefoInserir` vira atributo do objeto.

4. Nada de biblioteca pronta no lugar do algoritmo

Não usamos `HashMap`, `PriorityQueue`, `LinkedList` nem `Arrays.sort`: a função hash, o tratamento de colisão, a inserção ordenada e as ordenações estão escritos à mão nas duas linguagens. `List` só aparece como recipiente do resultado.

Diferença semântica mais importante: em C++, `Bolsa b = outra;` **copia** a struct. Em Java a variável guarda uma **referência** para o mesmo objeto. Como a mesma bolsa fica ao mesmo tempo na lista FEFO, no índice por id e nos vetores de relatório, tornamos `Bolsa` **imutável** (campos `final`). Assim, compartilhar a referência tem o mesmo efeito que copiar em C++, sem efeito colateral.

Compatibilidade ABO/Rh — matriz

A compatibilidade é uma **relação fixa** entre 8 tipos doadores e 8 tipos receptores, então a guardamos em uma matriz 8×8 indexada por `[doador][receptor]`. Responder "posso usar esta bolsa?" vira dois acessos a um vetor, sem nenhum `if` encadeado de regras.

Hemácias e plaquetas: o doador não pode levar um antígeno (A, B ou D/Rh) que o receptor não tenha; O− é o doador universal. **Plasma e crioprecipitado:** a lógica se inverte porque o plasma leva anticorpos; AB é o doador universal e o Rh não conta.

Hemácias / plaquetas

doador ↓ / receptor →	O-	O+	A-	A+	B-	B+	AB-	AB+
O-	✓	✓	✓	✓	✓	✓	✓	✓
O+	·	✓	·	✓	·	✓	·	✓
A-	·	·	✓	✓	·	·	✓	✓
A+	·	·	·	✓	·	·	·	✓
B-	·	·	·	·	✓	✓	✓	✓
B+	·	·	·	·	·	✓	·	✓
AB-	·	·	·	·	·	·	✓	✓
AB+	·	·	·	·	·	·	·	✓

Plasma (PFC) e crio (CRIO)

doador ↓ / receptor →	O-	O+	A-	A+	B-	B+	AB-	AB+
O-	✓	✓	·	·	·	·	·	·
O+	✓	✓	·	·	·	·	·	·
A-	✓	✓	✓	✓	·	·	·	·
A+	✓	✓	✓	✓	·	·	·	·
B-	✓	✓	·	·	✓	✓	·	·
B+	✓	✓	·	·	✓	✓	·	·
AB-	✓	✓	✓	✓	✓	✓	✓	✓
AB+	✓	✓	✓	✓	✓	✓	✓	✓

As matrizes acima foram lidas diretamente do arquivo `rota_vital.cpp` ao gerar este PDF. Matriz didática: não substitui protocolo imuno-hematológico.

Em C/C++	Em Java	Raciocínio da tradução
`enum Tipo { O_NEG, ... }`	`enum TipoSanguineo { O_NEG("O-"), ... }`	Em C++ o enum já é um inteiro e serve de índice. Em Java o enum é um objeto: usamos `ordinal()` para obter o mesmo índice e o rótulo "O-" passa a morar dentro do enum (no C++ era um vetor `NOME_TIPO[]` paralelo).
`const int COMPAT[8][8]`	`private static final int[][]`	`const` global vira `static final` privado: a tabela pertence à classe e não pode ser alterada de fora.
`bool compativel(...)`	`public static boolean compativel(...)`	Não há estado por objeto, então o método é estático (classe utilitária com construtor privado).
`int doadoresCompativeis(..., Tipo saida[])`	`TipoSanguineo[] doadoresCompativeis(...)`	C++ preenche um vetor de saída e devolve quantos; Java devolve o próprio array, já do tamanho certo.

C/C++

```
bool compativel(Componente comp, Tipo doador, Tipo receptor) {
    if (comp == PFC || comp == CRIO) return COMPAT_PLASMA[doador][receptor] == 1;
    return COMPAT_HEMACIAS[doador][receptor] == 1;
}
```

Java

```
public static boolean compativel(Componente comp, TipoSanguineo doador,
TipoSanguineo receptor) {
    // C++: COMPAT_PLASMA[doador][receptor]  ->  Java: PLASMA[doador.ordinal()]
[receptor.ordinal()]
    if (comp == Componente.PFC || comp == Componente.CRIO)
        return PLASMA[doador.ordinal()][receptor.ordinal()] == 1;
    return HEMACIAS[doador.ordinal()][receptor.ordinal()] == 1;
}
```

C/C++

```c
int doadoresCompativeis(Componente comp, Tipo receptor, Tipo saida[]) {
    int n = 0;
    for (int doador = 0; doador < N_TIPOS; doador++)
        if (compativel(comp, (Tipo)doador, receptor)) saida[n++] = (Tipo)doador;
    return n;
}
```

Java

```java
public static TipoSanguineo[] doadoresCompativeis(Componente comp, TipoSanguineo
receptor) {
    TipoSanguineo[] tipos = TipoSanguineo.values();
    TipoSanguineo[] tmp = new TipoSanguineo[tipos.length];
    int n = 0;
    for (int doador = 0; doador < tipos.length; doador++)   // C++: for (int doador
= 0; doador < N_TIPOS; doador++)
        if (compativel(comp, tipos[doador], receptor)) tmp[n++] = tipos[doador];
    TipoSanguineo[] saida = new TipoSanguineo[n];
    for (int i = 0; i < n; i++) saida[i] = tmp[i];
    return saida;
}
```

Operação	Custo	Justificativa pela implementação
Consultar compatibilidade	**O(1)**	Um `if` escolhe a matriz e um acesso `[doador][receptor]` responde. Não há laço. As matrizes têm tamanho fixo (64 inteiros), portanto o espaço também é O(1).
Listar doadores de um receptor	**O(T) = O(1)**	Percorre as 8 linhas da coluna do receptor. T = 8 é constante do domínio — não cresce com o estoque.

Testes 17 casos: 4 de compatibilidade e 4 de incompatibilidade para hemácias (incluindo O+ → O− barrado pelo Rh), 3 sobre as colunas (O− só tem 1 doador; AB+ tem 8; B+ recebe de O−, O+, B−, B+), a conferência das **64 combinações** contra a regra de antígenos (27 compatíveis), 4 de plasma (AB+ → O− permitido; O+ → A+ barrado) e 1 de crioprecipitado.

Priorização FEFO — lista com inserção ordenada

FEFO (First Expired, First Out): a bolsa que vence primeiro sai primeiro. Escolhemos manter a lista **sempre ordenada**, inserindo cada bolsa já na posição certa. Pagamos um pouco mais na entrada (recebimento, que acontece uma vez) para que a saída (pedido urgente) e o descarte sejam imediatos.

Inserindo a bolsa 1007 (24/10) em [CH A+]



O laço avança enquanto a nova bolsa **não** vence antes do próximo nó; ao parar, religa dois ponteiros: `novo->prox = atual->prox` e `atual->prox = novo`. Empate de validade é desempatado pelo id, o que deixa a ordem determinística nas duas linguagens.

Por que as vencidas saem em O(v)

Como a lista está ordenada por validade, as bolsas vencidas são sempre um **prefixo** da lista. O descarte só remove pela cabeça e para na primeira bolsa válida — nunca percorre a lista toda. No cenário, a varredura diária descartou 1005, 1019 e 1012.

Em C/C++	Em Java	Raciocínio da tradução
`struct NoBolsa { ...; NoBolsa* prox; }`	`private static class No { ...; No prox; }`	O nó vira classe interna privada: ninguém fora da lista manipula ponteiros.
`NoBolsa* fefoInserir(NoBolsa* head, Bolsa b)`	`void inserir(Bolsa b)`	Em C++ a função devolve a nova cabeça porque ela pode mudar. Em Java a cabeça é atributo, então o método é `void`.
`delete temp;`	`(sem equivalente)`	O coletor de lixo libera o nó quando `cabeca` deixa de apontar para ele.
`bool vemAntes(const Bolsa&, const Bolsa&)`	`a.venceAntesDe(b)`	Função livre vira método de instância. `const&` evitava cópia; em Java já é referência.

C/C++

```cpp
NoBolsa* fefoInserir(NoBolsa* head, Bolsa b) {
    NoBolsa* novo = new NoBolsa;
    novo->bolsa = b;
    novo->prox = NULL;
    if (head == NULL || vemAntes(b, head->bolsa)) {   // vira a nova cabeça
        novo->prox = head;
        return novo;
    }
    NoBolsa* atual = head;
    while (atual->prox != NULL && !vemAntes(b, atual->prox->bolsa))
        atual = atual->prox;
    novo->prox = atual->prox;                         // religa os ponteiros
    atual->prox = novo;
    return head;
}
```

Java

```java
public void inserir(Bolsa b) {
    No novo = new No(b);            // C++: NoBolsa* novo = new NoBolsa;
    if (cabeca == null || b.venceAntesDe(cabeca.bolsa)) {
        novo.prox = cabeca;         // C++: novo->prox = head; return novo;
        cabeca = novo;
    } else {
        No atual = cabeca;
        while (atual.prox != null && !b.venceAntesDe(atual.prox.bolsa))
            atual = atual.prox;     // C++: atual = atual->prox;
        novo.prox = atual.prox;
        atual.prox = novo;
    }
    tamanho++;
}
```

C/C++

```
NoBolsa* fefoRemoverPrimeiro(NoBolsa* head, Bolsa* removida) {
    *removida = head->bolsa;
    NoBolsa* temp = head;
    head = head->prox;
    delete temp;
    return head;
}
```

Java

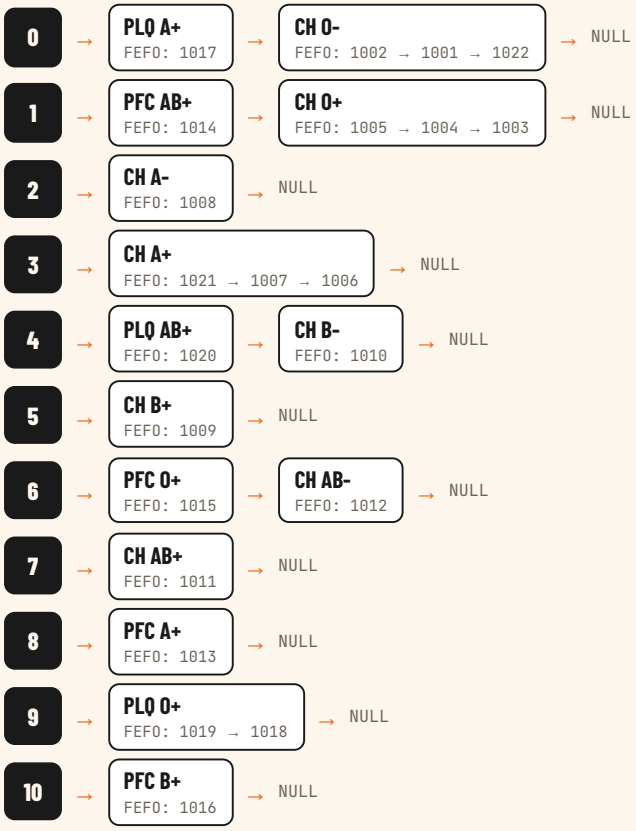
```
public Bolsa removerPrimeiro() {
    // C++: *removida = head->bolsa; temp = head; head = head->prox; delete temp;
    Bolsa b = cabeca.bolsa;
    cabeca = cabeca.prox;
    tamanho--;
    return b;
}
```

Operação	Custo	Justificativa pela implementação
Inserção ordenada	**O(m)**	m = bolsas daquela chave. Pior caso: a nova bolsa tem a maior validade e o `while` visita os m nós. Melhor caso O(1): ela vira a nova cabeça (primeiro `if`).
Retirar a que vence primeiro	**O(1)**	Só lê e troca a cabeça. A ordenação já foi paga na inserção.
Descartar vencidas	**O(v)**	v = vencidas. Remove pela cabeça enquanto `validade < hoje`; o laço para na primeira válida.

Índice de estoque — hash com encadeamento

Todo pedido pergunta "quais bolsas de **CH A+** eu tenho?". A chave é `componente × 8 + tipo` (4 componentes × 8 tipos = 32 chaves possíveis) e a função hash é `chave % 11`, os mesmos do HemoTrack. Cada entrada guarda a **lista FEFO** daquela combinação — o hash diz *onde* está e a lista já está na *ordem* certa.

Estado real da tabela após carregar as 22 bolsas



Gerado a partir da saída do programa. Com 11 posições (primo) e 15 chaves ocupadas há **4 colisões**, por exemplo CH O− (chave 0) e PLQ A+ (chave 11) no slot 0. Cada colisão é resolvida inserindo a nova entrada no início da cadeia.

Em C/C++	Em Java	Raciocínio da tradução
`struct HashEstoque { EntradaEstoque* slots[11]; }`	`private final Entrada[] slots = new Entrada[TAM_HASH];`	Java já inicializa o array com `null`, por isso `hashInicializar()` não precisa de laço; ele vira o construtor implícito.
`hashBuscar(HashEstoque* h, int chave)`	`buscar(int chave)`	O ponteiro `h` passa a ser o `this`. O `while (atual != NULL)` é idêntico com `null`.
`NoBolsa* fefo; int quantidade;`	`final ListaFefo fefo = new ListaFefo();`	Em C++ a entrada guarda a cabeça e o contador soltos; em Java guarda um objeto `ListaFefo` que sabe o próprio tamanho.
`(não usamos HashMap)`	`(não usamos HashMap)`	Usar `java.util.HashMap` esconderia exatamente o que a aula 06 pede: função hash e tratamento de colisão.

C/C++

```c
EntradaEstoque* hashBuscar(HashEstoque* h, int chave) {
    EntradaEstoque* atual = h->slots[funcaoHash(chave)];
    while (atual != NULL) {
        if (atual->chave == chave) return atual;
        atual = atual->prox;
    }
    return NULL;
}
```

Java

```java
public Entrada buscar(int chave) {
    Entrada atual = slots[funcaoHash(chave)];
    while (atual != null) {
        if (atual.chave == chave) return atual;
        atual = atual.prox;
    }
    return null;
}
```

C/C++

```c
EntradaEstoque* hashObterOuCriar(HashEstoque* h, int chave) {
    EntradaEstoque* e = hashBuscar(h, chave);
    if (e != NULL) return e;
    int indice = funcaoHash(chave);
    if (h->slots[indice] != NULL) h->colisoes++;
    e = new EntradaEstoque;
    e->chave = chave;
    e->fefo = NULL;
    e->quantidade = 0;
    e->prox = h->slots[indice];          // insere no início da cadeia
    h->slots[indice] = e;
    return e;
}
```

Java

```java
public Entrada obterOuCriar(int chave) {
    Entrada e = buscar(chave);
    if (e != null) return e;
    int indice = funcaoHash(chave);
    if (slots[indice] != null) colisoes++;
    e = new Entrada(chave);
    e.prox = slots[indice];          // insere no início da cadeia (igual ao C++)
    slots[indice] = e;
    return e;
}
```

Operação	Custo	Justificativa pela implementação
Buscar a lista de um tipo	**O(1 + α)**	Calcula `chave % 11` e percorre só a cadeia do slot. α = chaves/slots ≤ 32/11 ≈ 2,9, constante do domínio → O(1) na prática. Pior caso teórico O(chaves) se tudo colidisse no mesmo slot.
Receber bolsa	**O(m)**	O(1) no hash + O(m) da inserção ordenada na lista FEFO da chave.

Índice por id — hash com sondagem linear

Para "onde está a bolsa 1008?" usamos uma segunda tabela, agora com **endereçamento aberto**: um vetor de 101 posições (primo), `h(id) = id % 101`, e em caso de colisão testamos a próxima posição `(base + i) % 101`. Ter as duas técnicas no projeto é o "depende!" da aula 06 aplicado.

Por que duas técnicas diferentes?

Encadeamento no estoque: poucas chaves (32), cada uma com uma lista de tamanho variável e remoções a toda hora.

Sondagem linear no índice por id: uma bolsa por chave, fator de carga baixo (22/101 ≈ 22%, abaixo dos 70% recomendados) e memória contígua.

Remoção com marcação `REMOVIDO`

Se esvaziássemos a posição (`VAZIO`), uma busca posterior pararia ali e não acharia elementos que foram empurrados para frente por colisão. Por isso a remoção marca `REMOVIDO` : a busca "pula" a marca, e a inserção pode reaproveitá-la.

Análise crítica: agrupamento primário

Os ids sintéticos são sequenciais (1001…1022) e ocupam um bloco contínuo: posições 92–100 e 0–12. No cenário não há colisão, mas uma bolsa de id **1102** (1102 % 101 = 92) testaria 92, 93, …, 100, 0, …, 12 e só pararia na posição **13**: 23 sondagens.

–	–	1001	1002	1003	1004	1005	1006	1007	1008	1009	…	1020	1021	1022	1102	–
90	91	92	93	94	95	96	97	98	99	100		10	11	12	13	14

Preto = ocupado no cenário · laranja = onde o id 1102 cairia. Melhorias possíveis: hash duplo ou chave embaralhada antes do módulo.

Em C/C++	Em Java	Raciocínio da tradução
`void indiceInicializar(...) { for ... = VAZIO; }`	`Arrays.fill(chaves, VAZIO);`	Mesmo efeito, só que dentro do construtor.
`x->chave[pos] = REMOVIDO;`	`chaves[pos] = REMOVIDO;` `bolsas[pos] = null;`	Em Java também soltamos a referência à bolsa para o coletor de lixo poder liberá-la; em C++ a struct ficava no vetor sem custo extra.

C/C++

```
int indiceBuscar(IndiceId* x, int id) {
    int base = indiceHash(id);
    for (int i = 0; i < TAM_INDICE; i++) {
        int t = (base + i) % TAM_INDICE;
        if (x->chave[t] == VAZIO) return -1;     // nunca inserido
        if (x->chave[t] == id) return t;         // REMOVIDO é "pulado"
    }
    return -1;
}
```

Java

```
public int buscar(int id) {
    int base = hash(id);
    for (int i = 0; i < TAM_INDICE; i++) {
        int t = (base + i) % TAM_INDICE;
        if (chaves[t] == VAZIO) return -1;
        if (chaves[t] == id) return t;
    }
    return -1;
}
```

C/C++

```c
bool indiceRemover(IndiceId* x, int id) {
    int pos = indiceBuscar(x, id);
    if (pos == -1) return false;
    x->chave[pos] = REMOVIDO;   // marcação: não quebra a cadeia de sondagem
    return true;
}
```

Java

```java
public boolean remover(int id) {
    int pos = buscar(id);
    if (pos == -1) return false;
    chaves[pos] = REMOVIDO;
    bolsas[pos] = null;            // em Java, solta a referência para o GC
    return true;
}
```

Operação	Custo	Justificativa pela implementação
Buscar / inserir / remover	**O(1) médio**	Com fator de carga de 22%, quase sempre 1 acesso. Pior caso O(TAM) quando há agrupamento (exemplo do id 1102: 23 sondagens).

Fila de pedidos — FIFO encadeada

Requisições são atendidas na ordem de chegada. Mantemos ponteiros para **início** e **fim**: inserir no fim e remover do início custam O(1), sem deslocar elementos como aconteceria em um vetor.

C/C++

```cpp
void enqueue(FilaPedidos* f, Pedido p) {            // O(1): insere no fim
    NoPedido* novo = new NoPedido;
    novo->pedido = p;
    novo->prox = NULL;
    if (f->fim != NULL) f->fim->prox = novo;
    else f->inicio = novo;
    f->fim = novo;
    f->tamanho++;
}

bool dequeue(FilaPedidos* f, Pedido* saida) {       // O(1): remove do início
    if (f->inicio == NULL) return false;
    NoPedido* temp = f->inicio;
    *saida = temp->pedido;
    f->inicio = temp->prox;
    if (f->inicio == NULL) f->fim = NULL;
    delete temp;
    f->tamanho--;
    return true;
}
```

Java

```java
public void enqueue(Pedido p) {
    No novo = new No(p);
    if (fim != null) fim.prox = novo;
    else inicio = novo;
    fim = novo;
    tamanho++;
}

public Pedido dequeue() {
    if (inicio == null) return null;
    Pedido p = inicio.pedido;
    inicio = inicio.prox;
    if (inicio == null) fim = null;
    tamanho--;
    return p;
}
```

Em C/C++	Em Java	Raciocínio da tradução
`bool dequeue(FilaPedidos* f, Pedido* saida)`	`public Pedido dequeue()`	C++ devolve o sucesso em `bool` e o dado pelo ponteiro `saida`. Java não tem ponteiro de saída: devolve o próprio `Pedido`, ou `null` quando a fila está vazia.
`while (dequeue(&fila, &p))`	`while ((p = fila.dequeue()) != null)`	O laço do `main` muda de forma, mas faz a mesma coisa.
`Pedido p = {1, 3, A_POS, CH, 3, 90};`	`new Pedido(1, 3, A_POS, CH, 3, 90)`	A inicialização por chaves da struct vira construtor com os parâmetros na mesma ordem.

Operação	Custo	Justificativa pela implementação
enqueue	**O(1)**	Liga o novo nó em `fim->prox` e move `fim`. Nenhum laço.
dequeue	**O(1)**	Lê `inicio`, avança para `inicio->prox` e zera `fim` se a fila esvaziou.

No cenário

6 pedidos entram na ordem #1 (Gama) … #6 (Alfa) e são alocados exatamente nessa ordem. Cada pedido atendido recebe a sua rota na seção 11; os que não têm bolsas suficientes ficam pendentes.

Alocação integrada — o coração do sistema

Dado um pedido, o sistema precisa entregar **a bolsa compatível que vence primeiro**. É aqui que as estruturas se encontram: a **matriz** filtra os tipos, o **hash** leva a cada lista, o **FEFO** oferece a candidata de cada tipo na cabeça, e comparamos só essas cabeças.

Passo a passo do pedido #1 · Hospital Gama · CH A+ ×3

1. A coluna A+ da matriz diz que A+ recebe hemácias de **O−, O+, A− e A+**.
2. O hash devolve as quatro listas: [CH O−] 1002 → 1001 → 1022, [CH O+] 1004 → 1003 (1005 já foi descartada), [CH A−] 1008 e [CH A+] 1021 → 1007 → 1006. São 9 compatíveis para 3 pedidas, então o pedido pode ser atendido.
3. Três vezes compara só as cabeças e tira a que vence antes: **1004** (O+, 20/10), **1021** (A+, 21/10) e **1002** (O−, 22/10).

Regra de prioridade e por quê

É a regra da US04 do HemoTrack: entre **todos** os tipos compatíveis sai a bolsa que vence primeiro, o que reduz o descarte. E é **tudo ou nada**: sem bolsas suficientes, nenhuma sai e o pedido fica pendente. O custo dessa regra aparece no cenário: o pedido #1 gastou O+ e O− que venciam antes, e os pedidos #4 (O+ ×3) e #5 (B− ×2) ficaram **pendentes**. Priorizar o tipo idêntico pouparia o O−, mas deixaria vencer mais bolsas.

Em C/C++	Em Java	Raciocínio da tradução
`int alocar(Estoque* e, Pedido p, int hoje, Alocacao* out)`	`Alocacao alocar(Pedido p, int hoje)`	O estoque é injetado no construtor de `ServicoAlocacao` (o mesmo padrão de um `@Service` no Spring Boot) e o resultado volta como objeto, não por ponteiro.
`Bolsa bolsas[10]; int qtd;`	`List<Bolsa> bolsas`	Lista só como recipiente do resultado; `qtd` vira `size()`.
`Tipo doadores[8]; int nd = doadoresCompativeis(...)`	`TipoSanguineo[] doadores = ...`	O vetor + contador do C++ vira um array do tamanho exato.
`for (int i = 0; i < nd; i++)`	`for (TipoSanguineo t : doadores)`	O for-each percorre os mesmos doadores na mesma ordem.
`if (out->disponiveis < p.quantidade) return 0;`	`if (disponiveis < p.getQuantidade()) return out;`	Tudo ou nada: a checagem acontece antes de retirar qualquer bolsa.

C/C++

```c
int alocar(Estoque* e, Pedido p, int hoje, Alocacao* out) {
    out->pedido = p;
    out->qtd = 0;
    out->disponiveis = 0;
    if (p.quantidade < 1 || p.quantidade > MAX_POR_PEDIDO) return 0;

    // (a) coluna da matriz: doadores aceitos (8 consultas O(1))
    Tipo doadores[N_TIPOS];
    int nd = doadoresCompativeis(p.comp, p.receptor, doadores);

    // (b) uma busca O(1) no hash por doador; descarta as vencidas do início
    EntradaEstoque* listas[N_TIPOS];
    int nl = 0;
    for (int i = 0; i < nd; i++) {
        EntradaEstoque* en = hashBuscar(&e->porTipo, chaveEstoque(p.comp,
doadores[i]));
        if (en == NULL) continue;
        estoqueDescartarVencidas(e, en, hoje);      // vencidas nunca saem
        if (en->fefo != NULL) {
            listas[nl++] = en;
            out->disponiveis += en->quantidade;
        }
    }
    if (out->disponiveis < p.quantidade) return 0;  // tudo ou nada

    // (c) q vezes: compara as cabeças (no máx. 8) e retira a que vence antes
    while (out->qtd < p.quantidade) {
        EntradaEstoque* melhor = NULL;
        for (int i = 0; i < nl; i++) {
            if (listas[i]->fefo == NULL) continue;
            if (melhor == NULL || vemAntes(listas[i]->fefo->bolsa, melhor->fefo-
>bolsa))
                melhor = listas[i];
        }
        out->bolsas[out->qtd++] = estoqueRetirar(e, melhor);
    }
    return out->qtd;
}
```

Java

```java
public Alocacao alocar(Pedido p, int hoje) {
    Alocacao out = new Alocacao(p);
    if (p.getQuantidade() < 1 || p.getQuantidade() > MAX_POR_PEDIDO) return out;

    // (a) coluna da matriz: doadores aceitos (8 consultas O(1))
    TipoSanguineo[] doadores =
MatrizCompatibilidade.doadoresCompativeis(p.getComponente(), p.getReceptor());

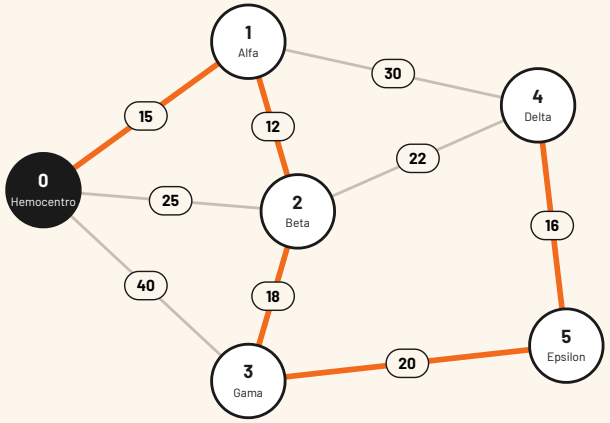
    // (b) uma busca O(1) no hash por doador; descarta as vencidas do início
    HashEstoque.Entrada[] listas = new HashEstoque.Entrada[doadores.length];   //
C++: EntradaEstoque* listas[N_TIPOS]
    int nl = 0;
    int disponiveis = 0;
    for (TipoSanguineo t : doadores) {                  // C++: for (int i = 0; i <
nd; i++)
        HashEstoque.Entrada en =
estoque.getPorTipo().buscar(HashEstoque.chave(p.getComponente(), t));
        if (en == null) continue;
        estoque.descartarVencidas(en, hoje);
        if (!en.getFefo().estaVazia()) {
            listas[nl++] = en;
            disponiveis += en.getFefo().tamanho();
        }
    }
    out.setDisponiveis(disponiveis);
    if (disponiveis < p.getQuantidade()) return out;    // tudo ou nada

    // (c) q vezes: compara as cabeças (no máx. 8) e retira a que vence antes
    while (out.quantidade() < p.getQuantidade()) {
        HashEstoque.Entrada melhor = null;
        for (int i = 0; i < nl; i++) {
            if (listas[i].getFefo().estaVazia()) continue;
            // C++: vemAntes(listas[i]->fefo->bolsa, melhor->fefo->bolsa)
            if (melhor == null ||
listas[i].getFefo().primeira().venceAntesDe(melhor.getFefo().primeira()))
                melhor = listas[i];
        }
        out.adicionar(estoque.retirar(melhor));
    }
    return out;
}
```

Operação	Custo	Justificativa pela implementação
Alocar um pedido	**O(v + q)**	Coluna da matriz (8 linhas) e no máximo 8 buscas O(1) no hash; descarte das v vencidas; para cada uma das q bolsas compara no máximo 8 cabeças e remove em O(1). Como T é constante, O(T + v + q·T) = O(v + q). **Não depende do tamanho n do estoque** — esse é o ganho de combinar hash e FEFO.

Grafo — matriz de adjacência

A rede tem **6 vértices** (hemocentro + 5 hospitais) e 9 estradas com tempo em minutos — escopo controlado, como pede a ficha. Guardamos o grafo como **matriz de adjacência** `ESTRADAS[u][v]` : o tempo da estrada direta, ou `INF` quando não há estrada. A rota (seção 11) só precisa perguntar "qual o tempo de u até v?", e a matriz responde em O(1).



Em laranja, as estradas usadas pelas rotas do cenário (seção 11).

Matriz de adjacência (min)

	0	1	2	3	4	5
0 Hemocentro	0	15	25	40	—	—
1 Alfa	15	0	12	—	30	—
2 Beta	25	12	0	18	22	—
3 Gama	40	—	18	0	—	20
4 Delta	—	30	22	—	0	16
5 Epsilon	—	—	—	20	16	0

— = não há estrada direta (`INF`). A matriz é simétrica porque as estradas são de mão dupla.

Em C/C++	Em Java	Raciocínio da tradução
`const int ESTRADAS[V][V] = {...};`	`private static final int[][] ESTRADAS`	Constante global vira constante privada da classe `Grafo`.
`const int adj[V][V] (parâmetro da rota)`	`new Grafo() / new Grafo(int[][] adj)`	Em C++ a rota recebe a matriz; em Java a matriz fica dentro de um objeto `Grafo`. O segundo construtor recebe outra matriz, usado nos testes (a matriz "beco").
`adj[u][v]`	`grafo.tempo(u, v)`	O acesso direto vira um método, que esconde como o grafo é guardado.
`const int INF = 1000000;`	`INF = 1_000_000;`	Valor alto que nunca é uma estrada real; a rota só segue `adj[u][v] < INF`.

Operação	Custo	Justificativa pela implementação
Tempo da estrada u–v	**O(1)**	Leitura de `adj[u][v]`.
Espaço	**$O(V^2)$**	Uma matriz V × V = 36 inteiros. Para uma malha pequena e com várias estradas, é mais simples que lista de adjacência.

C/C++

```cpp
const int ESTRADAS[V][V] = {
    //  0    1    2    3    4    5
    {   0,  15,  25,  40, INF, INF },   // 0 Hemocentro Central
    {  15,   0,  12, INF,  30, INF },   // 1 Hospital Alfa
    {  25,  12,   0,  18,  22, INF },   // 2 Hospital Beta
    {  40, INF,  18,   0, INF,  20 },   // 3 Hospital Gama
    { INF,  30,  22, INF,   0,  16 },   // 4 Hospital Delta
    { INF, INF, INF,  20,  16,   0 },   // 5 Hospital Epsilon
};
```

Java

```java
private static final int[][] ESTRADAS = {
    //  0    1    2    3    4    5
    {   0,  15,  25,  40, INF, INF },
    {  15,   0,  12, INF,  30, INF },
    {  25,  12,   0,  18,  22, INF },
    {  40, INF,  18,   0, INF,  20 },
    { INF,  30,  22, INF,   0,  16 },
    { INF, INF, INF,  20,  16,   0 },
};
```

Roteirização — vizinho mais próximo

Para cada pedido atendido, o caminhão sai do hemocentro e anda pela malha: em cada ponto vai para o **vizinho ainda não visitado com a estrada mais curta**, até chegar ao hospital. O caminho fica numa **pilha** (`caminho[]`). Se o caminhão cair num **beco sem saída** (nenhum vizinho novo), desempilha e tenta o próximo vizinho do ponto anterior. O vetor `visitado[]` garante que cada ponto entra na pilha uma vez só, então não há ciclo.

Em C/C++	Em Java	Raciocínio da tradução
`int rota(..., int caminho[], int* tamanho)`	`Rota vizinhoMaisProximo(origem, destino)`	Java não tem parâmetro de saída: caminho e tempo voltam juntos num objeto `Rota`.
`return -1; (sem ligação)`	`return null;`	A ausência de rota vira `null`.
`bool visitado[V]; for (...) visitado[i] = false;`	`boolean[] visitado = new boolean[n];`	Em Java o array já nasce com `false`.
`imprimirCaminho(caminho, tamanho)`	`rota.descricao()`	Devolve o texto em vez de imprimir, para reaproveitar no servidor web.

Resultado real no cenário

```
=== 4. ROTA DE CADA PEDIDO (vizinho mais proximo + janela + cadeia fria) ===
1. Pedido #1 em Hospital Gama | chegada  45 min | janela  90 | NO PRAZO | cadeia fria OK
   trajeto: Hemocentro Central -> Hospital Alfa -> Hospital Beta -> Hospital Gama
2. Pedido #2 em Hospital Alfa | chegada  15 min | janela  60 | NO PRAZO | cadeia fria OK
   trajeto: Hemocentro Central -> Hospital Alfa
3. Pedido #3 em Hospital Delta | chegada  81 min | janela  75 | ATRASO   | cadeia fria OK
   trajeto: Hemocentro Central -> Hospital Alfa -> Hospital Beta -> Hospital Gama -> Hospital Epsilon -> Hospital Delta
4. Pedido #6 em Hospital Alfa | chegada  15 min | janela  60 | NO PRAZO | cadeia fria OK
   trajeto: Hemocentro Central -> Hospital Alfa
Alertas: 1
Obs.: vizinho mais proximo e heuristica gulosa; nao garante o menor tempo nem a janela.
```

Limitação que o próprio sistema mostra

Até o Hospital Delta o algoritmo leva **81 min** (janela de **75**): de Alfa ele prefere Beta (12 min) à estrada direta para Delta (30 min), e depois segue pela estrada mais curta de cada ponto. Pela estrada Alfa → Delta seriam **45 min**. O algoritmo guloso olha só o próximo passo, e o sistema **detecta e sinaliza** o atraso. Um caminho mínimo exato (Dijkstra) evitaria isso, ao custo de uma fila de prioridade.

Operação	Custo	Justificativa pela implementação
Rota de um pedido	$O(V^2)$	Cada volta do `while` empilha um vértice novo ou desempilha um beco: no máximo 2V voltas. Em cada volta, o `for` varre a linha do vértice atual na matriz (V colunas). Total 2V × V.
Rotas do cenário	$O(k \cdot V^2)$	Uma rota por pedido atendido (k = 4 no cenário).

C/C++

```cpp
int rotaVizinhoMaisProximo(const int adj[V][V], int origem, int destino,
                           int caminho[], int* tamanho) {
    bool visitado[V];
    for (int i = 0; i < V; i++) visitado[i] = false;
    int topo = 0;
    caminho[topo++] = origem;
    visitado[origem] = true;
    while (topo > 0) {
        int atual = caminho[topo - 1];
        if (atual == destino) break;
        int vizinho = -1;
        for (int v = 0; v < V; v++)
            if (!visitado[v] && adj[atual][v] < INF &&
                (vizinho == -1 || adj[atual][v] < adj[atual][vizinho]))
                vizinho = v;
        if (vizinho == -1) topo--;                // beco sem saída: volta
        else { visitado[vizinho] = true; caminho[topo++] = vizinho; }
    }
    *tamanho = topo;
    if (topo == 0) return -1;                     // pilha vazia: sem ligação
    int tempo = 0;
    for (int i = 1; i < topo; i++) tempo += adj[caminho[i - 1]][caminho[i]];
    return tempo;
}
```

Java

```java
public Rota vizinhoMaisProximo(int origem, int destino) {
    int n = grafo.vertices();
    boolean[] visitado = new boolean[n];      // Java já inicia com false (C++: laço zerando)
    int[] pilha = new int[n];                 // C++: int caminho[] (recebido como parâmetro)
    int topo = 0;
    pilha[topo++] = origem;
    visitado[origem] = true;
    while (topo > 0) {
        int atual = pilha[topo - 1];
        if (atual == destino) break;
        int vizinho = -1;
        for (int v = 0; v < n; v++)
            if (!visitado[v] && grafo.tempo(atual, v) < Grafo.INF &&
                (vizinho == -1 || grafo.tempo(atual, v) < grafo.tempo(atual, vizinho)))
                vizinho = v;
        if (vizinho == -1) topo--;            // beco sem saída: volta
        else { visitado[vizinho] = true; pilha[topo++] = vizinho; }
    }
    if (topo == 0) return null;               // C++: return -1;
    int[] caminho = new int[topo];            // C++: *tamanho = topo;
    int tempo = 0;
    caminho[0] = pilha[0];
    for (int i = 1; i < topo; i++) {
        caminho[i] = pilha[i];
        tempo += grafo.tempo(pilha[i - 1], pilha[i]);
    }
    return new Rota(caminho, tempo);
}
```

Ordenação — os seis métodos do semestre

Os relatórios precisam do estoque restante em ordem FEFO. Implementamos os seis métodos sobre o mesmo vetor de bolsas, com o **mesmo critério** (`vemAntes`: validade, depois id) e um **contador de comparações** para medir o custo real.

Método	Pior caso	Melhor / médio	Comparações reais (n = 12)	Justificativa pelo código
Bubble Sort	$O(n^2)$	$O(n)$	51	Para com a flag `trocou` se uma passada não troca.
Selection Sort	$O(n^2)$	$O(n^2)$	66	O laço interno sempre vai até o fim: n(n−1)/2 = 66 com n = 12.
Insertion Sort	$O(n^2)$	$O(n)$	29	O `while` para cedo: o vetor vem quase ordenado, pois é exportado de listas FEFO.
Shell Sort	$O(n^2)$	$O(n \log n)$	34	Insertion Sort por gaps n/2, n/4…; no caso médio fica abaixo de n^2.
Quick Sort	$O(n^2)$	$O(n \log n)$	43	Lomuto com pivô = último; vetor já ordenado gera partições 0 / n−1.
Merge Sort	$O(n \log n)$	$O(n \log n)$	28	log n níveis × O(n) de intercalação; usa vetor auxiliar O(n). Estável.

Leitura dos números: o vetor exportado do estoque já vem **em blocos ordenados** (cada lista FEFO), então Insertion (29) e Merge (28) se saem melhor e o Selection (66) não tira proveito disso, exatamente como a teoria prevê.

C/C++ · Merge Sort

```cpp
void intercalar(Bolsa arr[], int esq, int meio, int dir) {
    int n = dir - esq + 1;
    Bolsa* aux = new Bolsa[n];                          // memória extra O(n)
    int i = esq, j = meio + 1, k = 0;
    while (i <= meio && j <= dir) {
        if (menor(arr[j], arr[i])) aux[k++] = arr[j++];
        else aux[k++] = arr[i++];                       // empate: esquerda
(estável)
    }
    while (i <= meio) aux[k++] = arr[i++];
    while (j <= dir) aux[k++] = arr[j++];
    for (k = 0; k < n; k++) arr[esq + k] = aux[k];
    delete[] aux;
}

void mergeSort(Bolsa arr[], int esq, int dir) {         // O(n log n) sempre
    if (esq >= dir) return;
    int meio = esq + (dir - esq) / 2;
    mergeSort(arr, esq, meio);
    mergeSort(arr, meio + 1, dir);
    intercalar(arr, esq, meio, dir);
}
```

Java · Merge Sort

```java
private void intercalar(Bolsa[] arr, int esq, int meio, int dir) {
    Bolsa[] aux = new Bolsa[dir - esq + 1];   // C++: new Bolsa[n] ... delete[] aux;
    int i = esq, j = meio + 1, k = 0;
    while (i <= meio && j <= dir) {
        if (menor(arr[j], arr[i])) aux[k++] = arr[j++];
        else aux[k++] = arr[i++];             // empate: esquerda -> estável
    }
    while (i <= meio) aux[k++] = arr[i++];
    while (j <= dir) aux[k++] = arr[j++];
    System.arraycopy(aux, 0, arr, esq, aux.length);   // C++: for (k...) arr[esq+k]
= aux[k];
}

private void mergeSort(Bolsa[] arr, int esq, int dir) {      // O(n log n) sempre
    if (esq >= dir) return;
    int meio = esq + (dir - esq) / 2;
    mergeSort(arr, esq, meio);
    mergeSort(arr, meio + 1, dir);
    intercalar(arr, esq, meio, dir);
}
```

Em C/C++	Em Java	Raciocínio da tradução
`Bolsa* aux = new Bolsa[n]; ... delete[] aux;`	`Bolsa[] aux = new Bolsa[...];`	Mesmo vetor auxiliar O(n); em Java não há `delete[]`.
`for (k...) arr[esq+k] = aux[k];`	`System.arraycopy(aux, 0, arr, esq, aux.length);`	Cópia equivalente, feita pela biblioteca padrão (não substitui o algoritmo, só copia).
`std::swap(arr[i], arr[j])`	`swap(arr, i, j)`	Java não passa elemento de array por referência; o método recebe o array e os dois índices.
`long long g_comparacoes (global)`	`private long comparacoes (atributo)`	Elimina estado global: cada objeto `Ordenacao` conta as próprias comparações.
`void quickSort(Bolsa arr[], int low, int high)`	`quickSort(Bolsa[] arr) + private quickSort(arr, low, high)`	Sobrecarga: o método público esconde os índices; o recursivo é privado.

Busca — sequencial, binária iterativa e recursiva

Pergunta do relatório: **"quantas bolsas vencem até daqui a 7 dias?"**. Com o vetor ordenado, basta achar o primeiro índice cuja validade passa do limite — esse índice *é* a quantidade. Fizemos a mesma busca binária nas duas formas da aula 02 para comparar.

C/C++ · iterativa

```cpp
int primeiroAposIterativa(Bolsa arr[], int n, int limite) {
    int inicio = 0, fim = n;                 // intervalo semiaberto [inicio, fim)
    while (inicio < fim) {
        int meio = inicio + (fim - inicio) / 2;
        if (arr[meio].validade <= limite) inicio = meio + 1;
        else fim = meio;
    }
    return inicio;
}
```

Java · iterativa

```java
public static int primeiroAposIterativa(Bolsa[] arr, int limite) {
    int inicio = 0, fim = arr.length;          // intervalo semiaberto [inicio, fim)
    while (inicio < fim) {
        int meio = inicio + (fim - inicio) / 2;
        if (arr[meio].getValidade() <= limite) inicio = meio + 1;
        else fim = meio;
    }
    return inicio;
}
```

C/C++ · recursiva

```cpp
int primeiroAposRecursiva(Bolsa arr[], int inicio, int fim, int limite) {
    if (inicio >= fim) return inicio;         // caso base
    int meio = inicio + (fim - inicio) / 2;
    if (arr[meio].validade <= limite)
        return primeiroAposRecursiva(arr, meio + 1, fim, limite);
    return primeiroAposRecursiva(arr, inicio, meio, limite);
}
```

Java · recursiva

```java
public static int primeiroAposRecursiva(Bolsa[] arr, int inicio, int fim, int
limite) {
    if (inicio >= fim) return inicio;          // caso base
    int meio = inicio + (fim - inicio) / 2;
    if (arr[meio].getValidade() <= limite)
        return primeiroAposRecursiva(arr, meio + 1, fim, limite);
    return primeiroAposRecursiva(arr, inicio, meio, limite);
}
```

Operação	Custo	Justificativa pela implementação
Sequencial por id	**O(n)**	Percorre o vetor até achar. Usada para comparar com o índice hash O(1).
Binária iterativa	**O(log n)**	O intervalo `[inicio, fim)` cai pela metade a cada volta. Espaço O(1).
Binária recursiva	**O(log n)**	Mesma divisão, mas cada chamada ocupa um quadro na pilha: espaço O(log n). Caso base: `inicio >= fim`.

Em C/C++	Em Java	Raciocínio da tradução
`Bolsa arr[], int n`	`Bolsa[] arr (arr.length)`	O parâmetro `n` some em Java.
`int meio = inicio + (fim-inicio)/2;`	idêntico	Mesma fórmula nas duas linguagens: evita estouro de `int` que `(inicio+fim)/2` poderia causar.

Verificação cruzada

No cenário, iterativa = **4** e recursiva = **4** (1017, 1020, 1007, 1009 vencem até 26/10). O teste automático confere que as duas batem com uma contagem sequencial.

Quadro-resumo de complexidade

Operação	Custo	Justificativa pela implementação
Compatibilidade ABO/Rh	**O(1)**	Acesso direto `[doador][receptor]` na matriz 8×8.
Inserção FEFO	**O(m)**	Percorre a lista até a posição; melhor caso O(1) ao virar cabeça.
Retirada FEFO	**O(1)**	Remove a cabeça.
Descarte de vencidas	**O(v)**	Vencidas são prefixo da lista ordenada.
Hash do estoque (encadeamento)	**O(1 + α)**	$\alpha \leq 32/11 \approx 2{,}9 \rightarrow$ constante.
Índice por id (sondagem linear)	**O(1) médio**	Fator de carga 22%; pior caso O(TAM) com agrupamento primário.
Fila FIFO	**O(1)**	Ponteiros de início e fim.
Alocação de um pedido	**O(v + q)**	Coluna da matriz e até 8 buscas no hash; q vezes compara até 8 cabeças. Independe de n.
Grafo (matriz de adjacência)	**O(1) / O(V^2)**	Tempo de uma estrada em O(1); a matriz ocupa V^2 posições.
Vizinho mais próximo (um pedido)	**O(V^2)**	No máx. 2V voltas (empilha ou desempilha), cada uma varre V colunas.
Bubble / Selection / Insertion	**O(n^2)**	Laços aninhados; Bubble e Insertion chegam a O(n) no melhor caso.
Shell / Quick / Merge	**O(n^2) / O(n^2) / O(n log n)**	Quick e Shell são O(n log n) no caso médio; Merge sempre O(n log n) com O(n) de memória extra.
Busca sequencial / binária	**O(n) / O(log n)**	Recursiva usa O(log n) de pilha.

Todas as justificativas citam a linha de código que gera o custo (o laço, o acesso direto, a recursão). Os números de comparações das ordenações e o estado do hash vêm da execução real, não de material genérico.

Dicionário geral de tradução C/C++ → Java

Em C/C++	Em Java	Raciocínio da tradução
`struct Bolsa { ... };`	`public final class Bolsa`	Dados com comportamento. Imutável porque em Java circula por referência.
`Bolsa novaBolsa(...)`	`new Bolsa(...)`	Função fábrica vira construtor.
`NoBolsa* prox; NULL`	`No prox; null`	Ponteiro vira referência.
`atual->prox`	`atual.prox`	Java não distingue objeto de ponteiro, então só existe o ponto.
`new NoBolsa / delete temp`	`new No(b) / (nada)`	O coletor de lixo libera a memória; some `estoqueLiberar()`.
`hashBuscar(HashEstoque* h, ...)`	`hashEstoque.buscar(...)`	O ponteiro para a estrutura vira o `this` implícito.
`hashInicializar(&h)`	`new HashEstoque()`	A inicialização vira construtor; arrays Java já nascem com `null`/0.
`head = fefoInserir(head, b)`	`lista.inserir(b)`	A cabeça vira atributo, e o método não precisa devolvê-la.
`bool dequeue(F* f, Pedido* saida)`	`Pedido dequeue()`	O parâmetro de saída vira valor de retorno (ou `null`).
`int rota(..., int caminho[], int* tamanho)`	`Rota vizinhoMaisProximo(...)`	Vários parâmetros de saída viram um objeto de resultado.
`enum Tipo + const char* NOME[]`	`enum com toString()`	O rótulo fica no próprio enum; o índice vem de `ordinal()`.
`Bolsa arr[], int n`	`Bolsa[] arr`	O array conhece o próprio tamanho (`length`).
`Bolsa bolsas[10]; int qtd;`	`List<Bolsa>`	Apenas como recipiente do resultado.
`std::swap(a, b)`	`swap(arr, i, j)`	Troca manual de referências no array.
`laço copiando struct a struct`	`arr.clone()`	Cópia rasa basta porque `Bolsa` é imutável.
`g_verboso, g_comparacoes, g_ok`	`atributos de Estoque, Ordenacao, TestesRotaVital`	Sem estado global: cada objeto tem o seu.
`const char* data(int) + buffers static`	`String Datas.formatar(int)`	Java devolve uma `String` nova com segurança, sem buffer estático.
`printf("%4lld", ...)`	`System.out.printf("%4d", ...)`	Mesmos formatos; `long` usa `%d` em Java.
`int main() com funções soltas`	`main() orquestrando objetos`	Estoque, ServicoAlocacao, Grafo e Roteirizador colaboram; no Spring Boot viram `@Service` + `@RestController`.

Testes e evidências de execução

38/38
testes passando em C++ e em Java

0
linhas diferentes entre as saídas (fora o cabeçalho com o nome da linguagem)

19
casos de compatibilidade/incompatibilidade (matriz + alocação)

Compatibilidade de hemacias (CH)

- ✔ CH: O- doa para AB+ (doador universal)
- ✔ CH: O- doa para O-
- ✔ CH: A- doa para A+
- ✔ CH: AB+ doa para AB+
- ✔ CH: O+ NAO doa para O- (Rh)
- ✔ CH: A+ NAO doa para O+ (antigeno A)
- ✔ CH: B- NAO doa para A- (antigeno B)
- ✔ CH: AB+ NAO doa para A+
- ✔ CH: receptor O- so recebe de O- (1 doador)
- ✔ CH: receptor AB+ recebe de todos (8 doadores)
- ✔ CH: coluna B+ da matriz = O-, O+, B-, B+
- ✔ CH: matriz confere com a regra de antigenos (64 pares, 27 compativeis)

Compatibilidade de plasma (PFC)

- ✔ PFC: AB+ doa para O- (plasma universal)
- ✔ PFC: A- doa para O+
- ✔ PFC: O+ NAO doa para A+ (anti-A)
- ✔ PFC: B+ NAO doa para AB- (anti-A)
- ✔ CRIO: segue a regra do plasma

Lista FEFO (insercao ordenada)

- ✔ FEFO: ordem 2,4,1,3 (validade crescente, empate por id)

Hash do estoque e indice por id

- ✔ Hash: [CH O-] tem 2 bolsas e a cabeca e a 11 (vence antes)
- ✔ Hash: chave inexistente retorna NULL
- ✔ Hash: CH O- (chave 0) e PLQ A+ (chave 11) colidem no slot 0 e ambas sao achadas
- ✔ Indice: bolsa 12 encontrada por id
- ✔ Indice: bolsa 99 inexistente

Alocacao integrada (matriz + hash + FEFO)

- ✔ Alocacao: A+ recebe a compativel que vence antes (O- 11, nao a A+ 12)
- ✔ Alocacao: depois, A+ 12 (vence antes de O- 10)
- ✔ Alocacao: bolsa alocada sai do indice por id
- ✔ Alocacao: B- descarta a vencida (13); 1 compativel para 3 pedidas -> nada sai (tudo ou nada)
- ✔ Alocacao: B- x1 recebe O- (10)
- ✔ Alocacao: nenhuma bolsa incompativel foi entregue
- ✔ Alocacao: plasma O- aceita AB+ (regra invertida)
- ✔ Alocacao: sem estoque compativel -> 0 bolsas (PLQ A+ nao serve para O-)

Ordenacoes e buscas

- ✔ Ordenacao: os 6 algoritmos produzem a mesma ordem FEFO
- ✔ Busca binaria iterativa = recursiva = contagem sequencial (5)
- ✔ Busca sequencial: id 7 na posicao 6

Grafo e rota (vizinho mais proximo)

- ✔ Vizinho: Hemocentro -> Alfa -> Beta -> Gama = 45 min
- ✔ Vizinho e guloso: ate Delta da 81 min (via Alfa direto seriam 45)
- ✔ Vizinho: beco sem saida em 1 -> volta e vai por 2 (10 min)
- ✔ Vizinho: destino sem ligacao -> -1

Evidência da tradução: `diff cpp/saida_cpp.txt java/saida_java.txt` mostra só a primeira linha ("Unidade 2 (C++)" × "Unidade 2 (Java)"). Os dois programas tomam as mesmas decisões de alocação, descarte e rota.

17 CHECKLIST DA ENTREGA

Atendimento aos critérios e à regra geral

Critérios da Unidade 2

✔ **Compatibilidade ABO/Rh via matriz**, com testes de compatibilidade e de incompatibilidade (seções 04 e 16).

✔ **Integração completa em Java:** rota + validade + compatibilidade + consulta por hash no mesmo fluxo (`RotaVitalApp` , seção 09).

✔ **Complexidade correta e justificada** pela implementação real, com números da execução (seções 04–14).

✔ **Tradução comentada C/C++ ↔ Java:** bloco de cabeçalho em cada classe, comentário `// C++:` nas linhas alteradas, código lado a lado e tabelas de raciocínio (seções 03–15).

✔ Os quatro algoritmos centrais: roteirização (vizinho mais próximo), FEFO (inserção ordenada), hash e matriz — os mesmos usados no HemoTrack.

Regra geral do Projeto Integrador

"Uma entrega só em Java, sem a versão em C/C++ correspondente e sem a tradução comentada, não é aceita — mesmo que o código Java funcione."

✔ Versão C/C++ completa: `cpp/rota_vital.cpp`, compilando sem avisos com `-Wall -Wextra`.

✔ Versão Java correspondente: 19 classes em `java/src/rotavital`, mais um servidor local opcional (`rotavital.web`) que só reaproveita essas classes.

✔ Tradução comentada no código e neste documento.

✔ Mesma lógica nas duas: comprovada pelos mesmos 38 testes e pela saída idêntica.

✔ Mesmos algoritmos do HemoTrack (Spring Boot): matriz, hash `O(1)`, alocação "compatível que vence primeiro, tudo ou nada" e vizinho mais próximo.

✔ Raciocínio exposto: decisões de estrutura, limitações (agrupamento primário, rota gulosa) e melhorias possíveis.

Como executar · limites e cuidados

C/C++

```
cd rota_vital/cpp
g++ -std=c++17 -Wall -Wextra -o rota_vital rota_vital.cpp
./rota_vital          (Windows: rota_vital.exe)
```

Java (JDK 11+)

```
cd rota_vital/java
javac -encoding UTF-8 -d out src/rotavital/*.java      (Windows: src\rotavital\*.java)
java -cp out rotavital.RotaVitalApp       # cenário + testes
java -cp out rotavital.TestesRotaVital    # só os testes
```

Servidor local (Java 15+)

```
javac -encoding UTF-8 -d out src/rotavital/*.java src/rotavital/web/*.java
java -cp out rotavital.web.ServidorRotaVital    # abrir http://localhost:8080
```

O pacote `rotavital.web` usa só o servidor HTTP do JDK e **não contém algoritmo**: chama as mesmas classes do console, que continuam idênticas à tradução do C++. O cenário de exemplo dá na página os mesmos resultados do console.

VS Code

Abra a pasta `rota_vital/java` como raiz, ou mantenha `rota_vital` aberta: o arquivo `.vscode/settings.json` incluído aponta `java/src` como código-fonte. Depois rode **Java: Clean Java Language Server Workspace**.

Limites e cuidados de escopo

- ✔ **Apenas dados sintéticos**: nenhuma informação real de doadores ou pacientes (LGPD).
- ✔ **Matriz ABO/Rh didática**: não substitui protocolo clínico nem prova cruzada.
- ✔ **Telemetria simulada**: a cadeia fria é um limite fixo de 120 min, sem sensor real.
- ✔ **Grafo limitado**: 6 vértices e 9 arestas, como pede a ficha.
- ✔ **Sem integração** com sistemas oficiais da Hemorrede.

Próximos passos

Os algoritmos já rodam no HemoTrack (Spring Boot, `RequisicaoService.atender`) com testes JUnit 5. Próximos passos: rota que priorize janelas apertadas; comparar o vizinho mais próximo com um caminho mínimo exato para medir a perda; hash duplo no índice por id.